

Chain Reaction: Nexus Protocol — Project Update Report

Md. Tahmidur Rahman Nafees (2022454642) · Sakib Rahman Rohan (2011350042)

tahmidur.nafees@northsouth.edu · rahman.rohan@northsouth.edu

Department of Electrical and Computer Engineering, North South University

Submitted to: Mohammad Shifat-E-Rabbi <rabbi.mohammad@northsouth.edu>

Update Report · Status as of June 13, 2026 · Repository: github.com/tahmidnafees619/ChainReaction

Abstract—This update report describes the state of the *Chain Reaction: Nexus Protocol* project at its first development milestone: a working, playable implementation of the core game rules, a rule-based computer opponent, and a themed browser frontend with basic audio. We summarize the architecture chosen, the rules implemented, the initial (single-strategy) AI, and a preliminary unit test suite, and outline the specific weaknesses identified at this stage — a computer opponent with no difficulty options, no browser-level verification, and no formalized robustness guarantees — that motivated the substantially expanded system described in the final report.

Index Terms—project status, game implementation, milestone report

I. Progress Summary

At this milestone, the project consists of a working, playable build of the core game: a headless rules engine (`ChainReactionEngine.js`), a first-pass computer opponent, and a themed HTML5 Canvas frontend ("Nexus Protocol"), plus an initial hand-rolled unit test suite for the engine. The system already satisfies the project's foundational architectural goal — a strict separation between game rules and rendering, communicated only through an explicit event interface, shown in Fig. 1 — which every later addition has continued to respect. This separation was a deliberate up-front decision rather than an incidental one: it is what makes it possible for the engine's rule correctness to be unit-tested under plain Node.js, with no browser and no rendering technology involved at all, and it is what will let the frontend and the AI evolve independently of one another in later milestones.

A. Rules Engine

The engine implements the full ruleset targeted for this project: a configurable $R \times C$ grid (2–10 players), position-dependent critical mass (corner/edge/interior cells hold 2/3/4 orbs respectively), wave-based simultaneous cascade resolution on explosion, a first-round-gated elimination rule, and win detection. It exposes state exclusively through a documented event system (`state_change`, `explosion`, `turn_change`, `player_eliminated`, `game_over`, `error`) and a single mutating entry point, `handlePlayerClick(row, col)`, with no dependency on the DOM. An initial suite of hand-rolled assertions (run under plain Node.js, no test framework) verifies construction validation, capacity computation, single and multi-cell explosions, turn rotation, the elimination guard, and a full small-board game played to a win. Table I summarizes coverage at this stage; all listed assertions pass at time of writing.

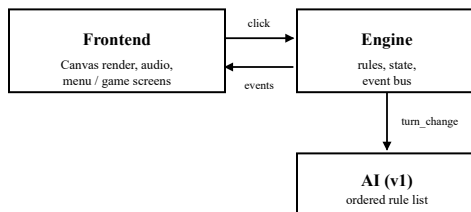


Fig. 1. Architecture at this milestone. The frontend and the version-1 rule-based AI are both consumers of the engine's public event interface; neither has any privileged access to engine internals.

TABLE I
Test Coverage at This Milestone

Area	Covered	Environment
Config validation	rows/cols/player-count bounds	Node.js
Capacity rule	corner/edge/interior cases	Node.js
Explosions	single-cell and chained	Node.js
Turn / elimination	rotation, first-round guard	Node.js
Full game	small board, played to a win	Node.js
Frontend / UI	<i>not yet covered</i>	—

B. Initial AI

A first computer opponent (`ChainReactionAI.js`) is implemented as a short ordered rule list: detonate an already-critical owned cell if one exists, otherwise prefer an open corner, otherwise prefer an interior cell, otherwise move at random. It is wired into the frontend via the engine's `turn_change` event, with a short artificial delay before it plays so a bot's move does not feel instantaneous. Any player slot can be toggled between human and bot control from the setup menu, and multiple simultaneous bot players are supported.

C. Frontend

The frontend renders the board on a 2-D canvas with a cyberpunk visual theme (a glassmorphic setup menu, animated grid background, per-player neon color palette) and includes an initial pass at procedurally synthesized audio via the Web Audio API — placement and explosion sound effects, an ambient menu tone, and a victory arpeggio — with no audio assets shipped or fetched. Player setup supports configurable grid dimensions and 2–10 operative slots.

II. Known Limitations at This Stage

Three specific gaps are identified at this milestone as the priorities for the next phase of work, listed here because each directly shaped a concrete deliverable in the final report:

1. **Single AI strength.** The rule-list opponent has no configurable difficulty and no mechanism for evaluating the actual consequence of a candidate move before playing it — it reacts to the immediate board rather than simulating outcomes. This is flagged as the highest-priority follow-up.
2. **No browser-level verification.** The test suite at this stage exercises only the DOM-free engine; no automated check yet exists for the rendered, interactive page itself, meaning defects in layout, event wiring, or UI state are currently only caught by manual inspection.
3. **No formalized robustness bound.** The cascade resolver has no explicit termination guarantee for pathological board

configurations, and asynchronous UI flows (such as a bot's "thinking" delay) have no protection against a user action changing game state while that delay is in flight.

III. Planned Next Steps

The next phase of work is planned to address each limitation in Section II directly: replacing the rule-list opponent with a heuristic evaluator that simulates each candidate move's actual cascade outcome before scoring it, and exposing this as multiple selectable difficulty tiers rather than a single fixed strategy; introducing a second, browser-driven test tier to verify real interactive behavior rather than engine logic alone; and adding explicit termination and state-consistency guarantees to the cascade resolver and the asynchronous bot-turn flow. The final report documents the completed versions of all three, including a 300-game empirical evaluation of the resulting difficulty tiers and a discussion of three specific defects that the new browser test tier was able to catch and the original engine-only tier could not.

Acknowledgment

Portions of this project's code, tests, and documentation were produced with the assistance of Anthropic's Claude (Claude Sonnet 5), an AI coding assistant, used interactively under the authors' direction and review; see reference [3] below and the accompanying "Use of AI Tools" note. A complete, expanded reference list appears in the final report.

References

- [1] Mozilla Developer Network, "Canvas API," MDN Web Docs. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API
- [2] Mozilla Developer Network, "Web Audio API," MDN Web Docs. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API
- [3] Anthropic, "Claude Sonnet 5" [Large language model], used as an AI coding assistant for implementation, testing, and documentation. [Online]. Available: <https://www.anthropic.com/claude>